

Java Backend Architecture

Interview Series

Chapter 13.6

Profiling & Performance Analysis

50+ Interview Q&A

Code Examples

Architecture Diagrams

Advanced Topics

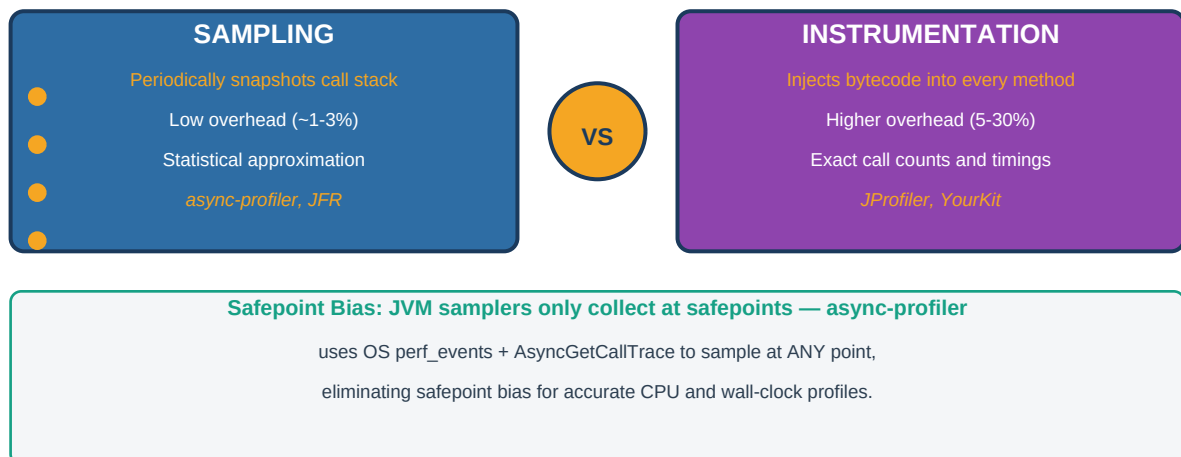
Profiling and Performance Analysis

Profiling is the systematic process of measuring where a Java application spends CPU time, allocates memory, contends on locks, and pauses for garbage collection. Unlike load testing which measures macro-level throughput and latency, profiling provides method-level visibility into hot paths, allocation hotspots, and thread contention — enabling engineers to direct optimization effort precisely where it matters. Modern Java profilers fall into two categories: sampling profilers that periodically capture the call stack with low overhead, and instrumentation profilers that inject bytecode into every method entry and exit for exact counts at higher overhead.

The JVM ecosystem offers a rich toolset: `async-profiler` provides low-overhead CPU, allocation, and wall-clock profiling without safepoint bias by using OS-level `perf_events` and `AsyncGetCallTrace`. Java Flight Recorder (JFR) — open-source since JDK 11 — continuously records hundreds of JVM events with sub-1% overhead. Commercial tools like JProfiler and YourKit offer deep heap analysis and JDBC/Spring integration. Flame graphs visualise call stack profiles where the x-axis is alphabetical (not time), the y-axis is call depth, and wide frames indicate hot code. Spring Boot Actuator and Micrometer provide production-grade metrics endpoints that complement offline profiling data.

Sampling vs Instrumentation

Sampling vs Instrumentation Profiling



Key Definitions

Sampling Profiler	Interrupts the JVM at a fixed interval and records the current thread call stack. Low overhead (~1-3%); statistical approximation of CPU usage.
Instrumentation Profiler	Injects bytecode at every method entry/exit to measure exact invocation counts and elapsed time. Higher overhead (5-30%) but precise.
async-profiler	Open-source low-overhead profiler using <code>AsyncGetCallTrace</code> + OS <code>perf_events</code> . Supports CPU, wall-clock, alloc, and lock modes without safepoint bias.
Safepoint Bias	Distortion in JVM-based samplers that collect stacks only at JVM safepoints. Methods preventing safepoints are under-represented in profiles.

Flame Graph	SVG visualization of aggregated stack traces. X-axis = alphabetical sort (not time). Y-axis = call depth. Width = proportion of total time.
JFR	Java Flight Recorder. Built-in JVM subsystem recording CPU, GC, allocation, lock, thread, and I/O events with less than 1% overhead.
JMC	Java Mission Control. Eclipse-based GUI for opening and analysing .jfr recordings with method profiling, memory, and GC tabs.
Wall-clock Mode	Profiling mode recording real elapsed time per method including blocking (I/O, sleep, lock waits). Shows latency bottlenecks.
CPU Mode	Profiling mode recording only on-CPU time; threads blocked on I/O or locks are excluded. Reveals compute bottlenecks.
Alloc Profiling	Records object allocation sites weighted by bytes allocated. Identifies object churn and GC pressure sources.
Lock Contention	Profiling mode capturing lock acquisition delays — identifies contested monitors and java.util.concurrent locks.
Spring Actuator	Spring Boot module exposing /actuator endpoints: /metrics (Micrometer), /health, /threaddump, /heapdump, /env.
Micrometer	Vendor-neutral application metrics facade (like SLF4J for metrics). Supports Prometheus, Datadog, CloudWatch, InfluxDB backends.
perf_events	Linux kernel subsystem exposing hardware PMU counters (CPU cycles, cache misses, branch mispredictions) used by async-profiler.
Method Profiling	Measurement of individual method execution frequency and duration. Foundation of hotspot identification for targeted optimization.

Flame Graph Anatomy

Flame Graph Anatomy

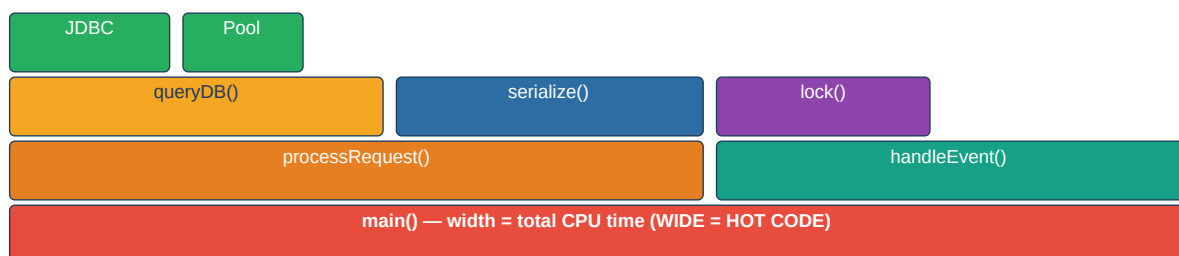
Colors = arbitrary (may highlight package or type)

Flat tops = leaf methods where CPU actually burns

Wide frames = methods consuming most CPU time

X-Axis = Alphabetical order (NOT time order)

Y-Axis = Call Stack Depth (bottom=root, top=leaf)



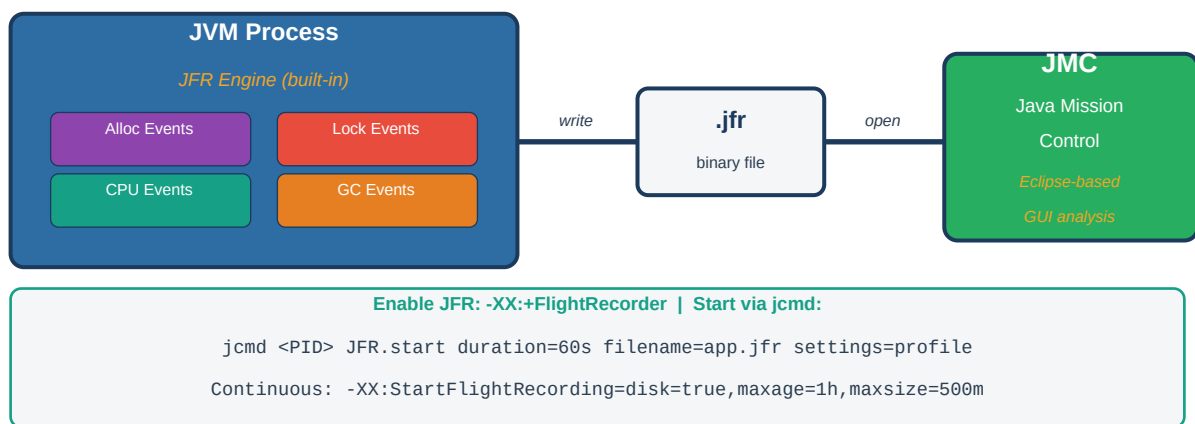
Tool Comparison: Profilers

Tool	Type	Overhead	Safepoint Free	Alloc Profiling	Best For
------	------	----------	----------------	-----------------	----------

async-profiler	Sampling	1-3%	Yes	Yes (TLAB)	CI/CD profiling, flame graphs
JFR	Sampling+Events	<1%	Partial	Yes	Production always-on recording
JProfiler	Instrumentation	5-20%	No	Yes (deep)	Development deep-dive analysis
YourKit	Both	3-15%	No	Yes	Memory leak and thread analysis
IntelliJ Profiler	Sampling	2-5%	Partial	Yes	IDE-integrated dev profiling
Micrometer/Actuator	Metrics	<0.1%	N/A	No	Production operational metrics

JFR and JMC Architecture

JFR and JMC Architecture



Annotated Code Examples

1. async-profiler — command-line usage

```
# CPU profiling for 30 seconds, output flame graph SVG ./profiler.sh -e cpu -d 30 -f
/tmp/cpu_flame.svg <PID> # Allocation profiling – sample every 512KB allocated ./profiler.sh -e
alloc -d 30 -f /tmp/alloc_flame.svg <PID> # Wall-clock mode – includes blocking time (I/O,
sleep, lock) ./profiler.sh -e wall -d 30 -f /tmp/wall_flame.svg <PID> # Lock contention
profiling ./profiler.sh -e lock -d 30 -f /tmp/lock_flame.svg <PID> # Output JFR format
(compatible with JMC) ./profiler.sh -e cpu -d 60 -f /tmp/profile.jfr <PID>
```

2. async-profiler — Java agent API

```
// Startup: -agentpath:/path/to/libasyncProfiler.so=start,event=cpu,file=profile.jfr import
one.profiler.AsyncProfiler; AsyncProfiler profiler = AsyncProfiler.getInstance(); // Start CPU
profiling profiler.execute("start,event=cpu,interval=1ms"); // ... run workload ... // Stop and
dump profiler.execute("stop,file=/tmp/profile.jfr"); // Allocation profiling with TLAB
sampling (very low overhead)
profiler.execute("start,event=alloc,alloc=512k,file=/tmp/alloc.jfr");
```

3. Java Flight Recorder — JVM flags and jcmd

```
// JFR is enabled by default since JDK 11 // Start recording via jcmd jcmd <PID> JFR.start
name=MyRecording duration=60s filename=/tmp/app.jfr settings=profile // Dump current
continuous recording jcmd <PID> JFR.dump name=MyRecording filename=/tmp/snapshot.jfr //
Always-on continuous recording //
```

```
-XX:StartFlightRecording=disk=true,maxage=1h,maxsize=500m,filename=app.jfr // Custom JFR event
import jdk.jfr.*; @Name("com.myapp.SlowQuery") @Label("Slow Database Query")
@Category("Database") public class SlowQueryEvent extends Event { @Label("SQL") public String
sql; @Label("Duration MS") public long durationMs; } SlowQueryEvent evt = new SlowQueryEvent();
evt.begin(); // ... execute query ... evt.durationMs = elapsed; evt.sql = sql; evt.commit(); //
only recorded if event is enabled in recording
```

4. Spring Boot Actuator and Micrometer

```
# application.yml management: endpoints: web: exposure: include:
health,info,metrics,prometheus,threaddump,heapdump metrics: distribution:
percentiles-histogram: http.server.requests: true percentiles: http.server.requests:
0.5,0.95,0.99 // Custom Micrometer timer in a Spring service @Service public class OrderService
{ private final Counter orderCounter; private final Timer orderTimer; public
OrderService(MeterRegistry registry) { this.orderCounter = registry.counter("orders.created",
"status", "success"); this.orderTimer = registry.timer("orders.processing.time"); } public
Order createOrder(OrderRequest req) { return orderTimer.record(() -> { Order order =
processOrder(req); orderCounter.increment(); return order; }); } }
```

5. JFR event streaming for lock contention (JDK 14+)

```
import jdk.jfr.consumer.*; try (var rs = new RecordingStream()) {
rs.enable("jdk.JavaMonitorWait") .withThreshold(Duration.ofMillis(10));
rs.enable("jdk.JavaMonitorEnter") .withThreshold(Duration.ofMillis(5));
rs.onEvent("jdk.JavaMonitorWait", event -> { System.out.printf("Thread %s waited on %s for
%.1fms\n", event.getThread("eventThread").getJavaName(),
event.getClass("monitorClass").getName(), event.getDuration("duration").toMillis()); });
rs.onEvent("jdk.JavaMonitorEnter", event -> { System.out.printf("Contention: %s on %s
(%.1fms)\n", event.getThread("eventThread").getJavaName(),
event.getClass("monitorClass").getName(), event.getDuration("duration").toMillis()); });
rs.startAsync(); Thread.sleep(60_000); }
```

6. Allocation profiling via JFR ObjectAllocationInNewTLAB

```
import jdk.jfr.consumer.*; import java.util.*; import java.util.concurrent.atomic.*;
Map<String, AtomicLong> allocSites = new ConcurrentHashMap<>(); try (var rs = new
RecordingStream()) { rs.enable("jdk.ObjectAllocationInNewTLAB");
rs.enable("jdk.ObjectAllocationOutsideTLAB"); rs.onEvent("jdk.ObjectAllocationInNewTLAB",
event -> { String klass = event.getClass("objectClass").getName(); long bytes =
event.getLong("tlabSize"); allocSites.computeIfAbsent(klass, k -> new AtomicLong())
.addAndGet(bytes); }); rs.start(); Thread.sleep(30_000); } // Print top 10 allocation sites by
bytes allocSites.entrySet().stream() .sorted(Map.Entry.<String,AtomicLong>comparingByValue(
Comparator.comparingLong(AtomicLong::get)).reversed()) .limit(10) .forEach(e ->
System.out.printf("%10d bytes %s\n", e.getValue().get(), e.getKey()));
```

Interview Q&A;

Q1. What is the difference between a sampling and an instrumentation profiler?

A sampling profiler periodically interrupts the JVM and records the current thread call stack (e.g., every 1ms). Overhead is low (1-3%) but the result is a statistical approximation. An instrumentation profiler injects bytecode at every method entry and exit, giving exact invocation counts and timings at higher overhead (5-30%). Sampling is preferred for production; instrumentation for development deep-dives.

Q2. What is safepoint bias and why does it matter for profiling?

Safepoint bias occurs because most JVM-based sampling profilers can only collect stack traces when the JVM reaches a safepoint. Methods that delay safepoints are under-represented, making them appear faster

than they are. `async-profiler` eliminates this by using OS-level signals (`perf_events` on Linux) and the `AsyncGetCallTrace` API to sample at any point, not just safepoints.

Q3. How does `async-profiler` work internally?

`async-profiler` uses the OS `perf_events` subsystem to receive a `SIGPROF` signal at a configurable interval. Upon the signal, it calls `AsyncGetCallTrace` (a non-public JVM API) to walk the Java call stack without needing a safepoint. For allocation profiling it instruments TLAB slow paths. For lock profiling it intercepts monitor enter/wait events.

Q4. What do the axes of a flame graph represent?

The X-axis is alphabetical ordering of stack frames — NOT time order. The Y-axis is call stack depth from bottom (root) to top (leaf). The width of each frame is the proportion of total CPU samples spent in that call path. Wide flat-topped frames are hot methods and prime optimization targets.

Q5. What is Java Flight Recorder and what overhead does it have?

JFR is a built-in JVM profiling subsystem recording CPU, GC, allocation, lock, thread, I/O, and classloading events using a circular binary buffer. Overhead is below 1% in default configuration, making it safe for always-on production use. It writes `.jfr` binary files analysed by JMC.

Q6. How does JFR differ from `async-profiler`?

JFR is a JVM-native continuous event recording system covering broad event types with very low overhead; it has some safepoint bias for method profiling. `async-profiler` is a focused, safepoint-bias-free profiler optimized for high-resolution CPU/allocation/wall-clock flame graphs. They complement each other: JFR for always-on production recording, `async-profiler` for targeted profiling sessions.

Q7. What are the different profiling modes of `async-profiler`?

CPU mode: on-CPU time via `perf_events` (excludes blocked threads). Wall-clock mode: real elapsed time including blocking — best for latency analysis. Alloc mode: object allocations via TLAB instrumentation — finds object churn. Lock mode: monitor contention events. Perf mode: Linux hardware counters (CPU cycles, cache misses, branch mispredictions).

Q8. How do you identify memory allocation hotspots?

Use `async-profiler` in alloc mode or JFR `ObjectAllocationInNewTLAB` events. Both sample allocations to Thread-Local Allocation Buffers weighted by size, producing flame graphs showing which call sites create the most objects by bytes. High allocation sites increase GC pressure. Fixes include object pooling, buffer reuse, avoiding autoboxing, and redesigning data structures.

Q9. What Spring Boot Actuator endpoints are useful for performance analysis?

`/actuator/metrics` — Micrometer metrics including JVM memory, GC count/time, thread counts, HTTP latency percentiles. `/actuator/prometheus` — Prometheus scrape endpoint. `/actuator/threaddump` — live thread dump for deadlock detection. `/actuator/heapdump` — downloads an HPROF heap dump. `/actuator/env` — active configuration.

Q10. How do you configure Micrometer to expose HTTP latency percentiles?

In `application.yml` set `management.metrics.distribution.percentiles-histogram.http.server.requests=true` and `management.metrics.distribution.percentiles.http.server.requests=0.5,0.95,0.99`. This publishes p50/p95/p99 for every request path. Use `percentiles-histogram=true` for Prometheus server-side computation; use `percentiles` for fixed app-side computation.

Q11. What is the difference between CPU profiling and wall-clock profiling?

CPU profiling records only time a thread is executing on a CPU core — threads blocked on I/O or locks are excluded. Wall-clock profiling records real elapsed time regardless of thread state. CPU profiling finds compute bottlenecks (hot algorithms, serialization). Wall-clock profiling finds latency bottlenecks (slow DB queries, network calls, lock waits).

Q12. How do you detect lock contention using JFR?

Enable `jdk.JavaMonitorEnter` (threshold ~5ms) and `jdk.JavaMonitorWait` events in a JFR recording. `JavaMonitorEnter` fires when a thread blocks trying to acquire an object monitor. `JavaMonitorWait` fires on `wait()` calls. In JMC the Lock Instances tab visualises contention. With JFR event streaming (JDK 14+) you can react to contention events in real time.

Q13. What profiling information does JMC provide?

JMC tabs include: Method Profiling (hot methods flame view and table), Memory (heap live set, allocation rate, TLAB stats), GC (pause times, causes, heap regions), Threads (thread states over time, deadlock detection), I/O (socket and file latencies), Lock Instances (monitor contention), and Environment (JVM flags, CPU/OS info).

Q14. How do you profile an application running in Docker?

Mount `async-profiler` into the container or include it in the image. For `perf_events`-based profiling the container needs `SYS_ADMIN` capability or `perf_event_open=1` on the host. Alternatively use JFR (no special privileges required) or `async-profiler` with `-e itimer` for `SIGPROF`-based fallback that works without `perf_events` privileges.

Q15. What is the significance of a flat-topped frame in a flame graph?

A flat-topped frame — with no visible child frames above it — is a leaf method where CPU time is actually consumed. Wide flat-topped frames are prime optimization targets. If many tall narrow stacks converge to a single wide lower frame, that lower frame is a common ancestor whose expensive children are the bottleneck.

Q16. How do you add a custom JFR event?

Extend `jdk.jfr.Event`, annotate the class with `@Name`, `@Label`, and `@Category`. Add public fields annotated with `@Label`. Call `evt.begin()` before the operation, populate fields, then call `evt.commit()`. When the event type is disabled in the recording configuration, `commit()` has zero overhead. The event appears in JMC under its category.

Q17. What is object churn and how does allocation profiling help?

Object churn is rapid creation and abandonment of short-lived objects, causing frequent minor GC collections. Even though allocation is cheap (bump pointer), high churn increases GC pause frequency and reduces CPU cache efficiency. Allocation flame graphs reveal the exact call sites responsible, enabling targeted fixes: object reuse, primitive arrays instead of boxed collections, pooling expensive objects.

Q18. What are perf events and how does async-profiler use them?

`perf_events` is the Linux kernel performance monitoring subsystem (`perf_event_open` syscall) providing hardware PMU counters and software events. `async-profiler` uses `perf_event_open` to configure CPU cycle overflow interrupts, delivering a signal to the profiled thread at the sampling interval, then calls `AsyncGetCallTrace` to walk the Java stack for an accurate, safepoint-bias-free profile.

Q19. How do you integrate profiling into a CI/CD pipeline?

Run `async-profiler` or JFR during performance regression tests triggered on each merge. Compare flame graphs or JFR metrics (alloc rate, GC pause, p99 latency) against a baseline. Store `.jfr` files or flame graph SVGs as build artifacts. Alert on regressions: if allocation rate increases over 20% or p99 latency degrades beyond an SLA threshold, fail the build. Use shell scripts or custom Maven plugins to automate.

Q20. What is the role of Micrometer in a microservices architecture?

Micrometer is the metrics instrumentation facade for Spring Boot, analogous to SLF4J for logging. It instruments JVM internals (GC, memory, threads), HTTP requests, data source connections, cache hit rates, and custom business metrics. Backends are pluggable: Prometheus (most common), Datadog, CloudWatch, InfluxDB, New Relic. Each service exposes `/actuator/prometheus`, Prometheus scrapes all services, and Grafana provides fleet-wide dashboards.